Two Column Reading Order

ORDERMARK 01. Every time a language model reads a web page, it pays for the whole page. The header pays. The cookie banner pays. The three-deep navigation menu pays, and so does the newsletter modal, the social share rail, and the footer that repeats the navigation menu a second time in smaller type.

ORDERMARK 02. The article itself is often a small minority of the bytes. On a typical content site the prose a reader actually came for accounts for a fraction of the delivered markup, and the rest is scaffolding that carries no meaning for a model trying to answer a question about the text.

ORDERMARK 03. This is why converting a page to Markdown appears to save so many tokens. The saving is real, but the mechanism is widely misattributed. Markdown syntax is not meaningfully cheaper than the equivalent HTML tags on a per-element basis. What actually happens is that the extraction step throws away the scaffolding, and the format conversion gets the credit.

ORDERMARK 04. The distinction matters because it tells you where to spend effort. If the win came from syntax, you would optimise the serialiser. Because the win comes from extraction, you should optimise the boilerplate detector, and you should measure extraction quality rather than output length.

ORDERMARK 05. It also tells you when the win will not appear. A page that is already mostly prose, such as a plain-text RFC or a documentation page rendered without a shell, has little scaffolding to remove. Running it through an extractor produces a small reduction and occasionally a regression, because aggressive extractors sometimes discard content they mistake for furniture.

ORDERMARK 06. There is a second, quieter effect that pushes in the opposite direction. Structure carries meaning. Headings tell a model how a document is organised; list markers tell it that items are peers; table pipes tell it which cell belongs to which column. Strip all of that in pursuit of a lower token count and you can end up with a cheaper prompt that produces worse answers.

ORDERMARK 07. The practical rule that falls out of this is short enough to remember. Strip the boilerplate, because it is pure cost. Keep the structure, because it is load-bearing. And measure both the token count and whether the answer is still correct, because a token count on its own is not a quality metric and was never meant to be one.

ORDERMARK 08. Every time a language model reads a web page, it pays for the whole page. The header pays. The cookie banner pays. The three-deep navigation menu pays, and so does the newsletter modal, the social share rail, and the footer that repeats the navigation menu a second time in smaller type.

ORDERMARK 09. The article itself is often a small minority of the bytes. On a typical content site the prose a reader actually came for accounts for a fraction of the delivered markup, and the rest is scaffolding that carries no meaning for a model trying to answer a question about the text.

ORDERMARK 10. This is why converting a page to Markdown appears to save so many tokens. The saving is real, but the mechanism is widely misattributed. Markdown syntax is not meaningfully cheaper than the equivalent HTML tags on a per-element basis. What actually happens is that the extraction step throws away the scaffolding, and the format conversion gets the credit.

ORDERMARK 11. The distinction matters because it tells you where to spend effort. If the win came from syntax, you would optimise the serialiser. Because the win comes from extraction, you should optimise the boilerplate detector, and you should measure extraction quality rather than output length.

ORDERMARK 12. It also tells you when the win will not appear. A page that is already mostly prose, such as a plain-text RFC or a documentation page rendered without a shell, has little scaffolding to remove. Running it through an extractor produces a small reduction and occasionally a regression, because aggressive extractors sometimes discard content they mistake for furniture.